Vision Extract

Isolation from Images using Image Segmentation

Submitted by: Chahat Agarwal

Guided by: Suman Sir

Project Overview

Project Statement: To build a machine learning model capable of automatically extracting the main subject from an image. For any given input picture, the model should output a new image in which only the subject is visible and the background is rendered completely black.

Goal: To automate the process of subject isolation, a critical task in photography, digital art, and augmented reality.

Applications:

- **E-commerce:** Automatically removing backgrounds from product photos.
- **Graphic Design:** Speeding up the process of creating cutouts for marketing materials.
- **Virtual Conferencing:** Creating virtual green screens for background replacement.
- **Photography:** Assisting photographers in post-production and editing.

Dataset Overview

Dataset: COCO (Common Objects in Context) 2017

- **Subset Used:** val2017, containing 5,000 diverse and complex images.
- **Annotations:** The dataset provides rich instance segmentation annotations in JSON format, defining the precise pixel-wise boundaries for 80 different object categories (person, car, dog, etc.).
- **Complexity:** The images feature multiple objects, occlusions, and varied lighting conditions, making it a challenging and realistic benchmark for training.

Methodology

- **Data Preparation:** We developed a robust data pipeline to load images and their corresponding annotations. This involved transforming the multi-object instance masks into a single binary foreground/background mask for our specific task.
- **Model Development & Training:** We started with a foundational U-Net model and progressively iterated to a powerful U-Net architecture with a pre-trained ResNet34 encoder. This model was fine-tuned on the COCO data.
- **Deployment:** The final, high-performance model was integrated into an interactive web application using Streamlit, providing a user-friendly interface for real-world use.

Data Preprocessing

To prepare the data for the model, several key steps were taken:

- **Image Resizing:** All input images were resized to a uniform 256x256 resolution to ensure consistent input dimensions for the model.
- **Mask Combination:** For each image, all individual object masks provided by the COCO dataset were merged into a single binary mask representing the entire foreground.
- **Data Augmentation:** To improve model generalization and prevent overfitting, we applied on-the-fly augmentations during training, including:
  - Random Horizontal Flips
  - Random Rotations
- **Tensor Conversion & Normalization:** Images were converted to PyTorch tensors, and pixel values were normalized using ImageNet's standard mean and standard deviation.

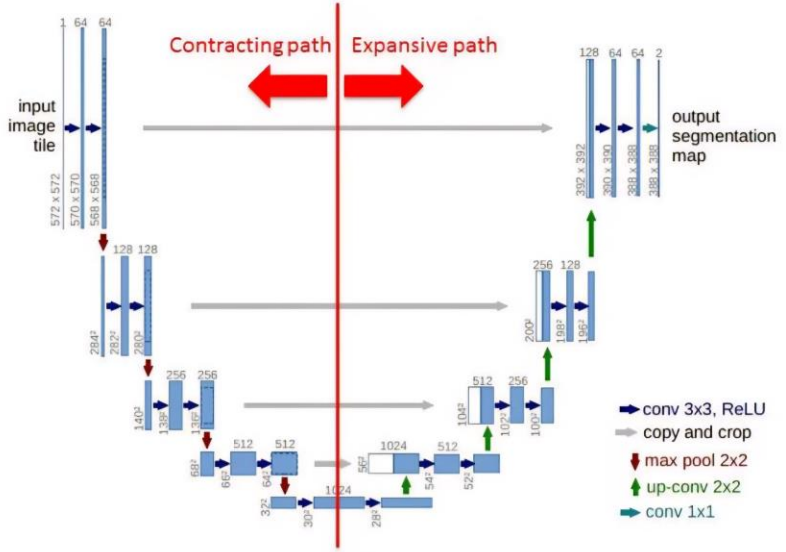
Feature Extraction

A core component of our final model is the **pre-trained encoder**.

- **Encoder:** We used a **ResNet34** architecture as the encoder for our segmentation model.
- **Pre-trained on ImageNet:** This ResNet34 was pre-trained on the massive ImageNet dataset (1.2 million images).
- **Benefit:** Instead of learning visual features from scratch, our model started with a powerful, built-in understanding of edges, textures, shapes, and objects. This technique, known as **Transfer Learning**, dramatically accelerates training and leads to significantly better performance.

Model Architecture

- Initial Model: A standard U-Net built from scratch. While it learned, it produced noisy and inaccurate visual results.
- Final Model: A U-Net from the segmentation-models-pytorch library that uses a pre-trained ResNet34 encoder. This architecture combines the excellent localization capabilities of the U-Net decoder with the powerful feature extraction of a pre-trained ResNet, resulting in clean and accurate segmentation masks.



Training and Evaluation

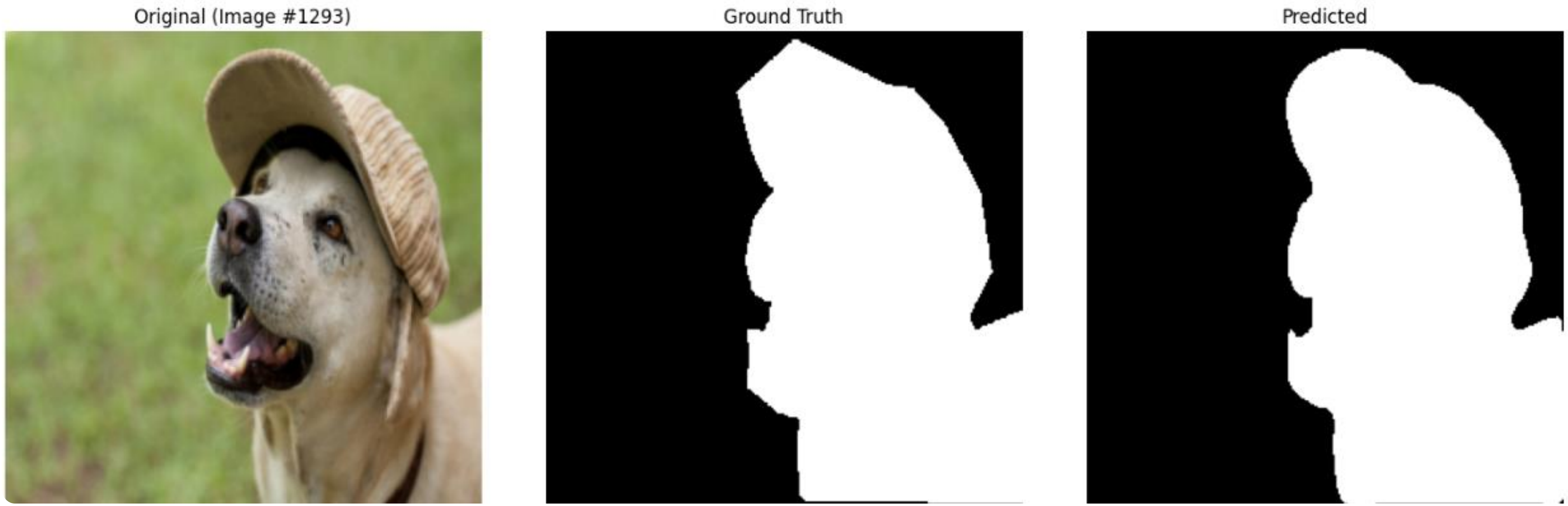
Optimizer: AdamW, an improved version of the Adam optimizer that often leads to better generalization.

Learning Rate: A low learning rate of 1e-4, suitable for fine-tuning a pre-trained model. A ReduceLROnPlateau scheduler was used to automatically decrease the rate if performance stagnated.

Loss Function: A **Combined Loss** (70% BCEWithLogitsLoss + 30% Dice Loss) was used. This balances pixel-wise accuracy with the geometric similarity of the predicted mask shape.

Evaluation Metrics:

- **Dice Coefficient:** Measures the overlap between the predicted and true masks.
- **Pixel Accuracy:** The percentage of correctly classified pixels.



Results

Quantitative Results:

- **Validation Dice Score:** Reached a peak of **0.9346 (93.5%)**
- **Validation Pixel Accuracy:** Reached a peak of **96.18%**

Qualitative (Visual) Improvement: The visual results improved dramatically from the initial scratch-trained U-Net to the final fine-tuned model, producing clean, accurate, and visually pleasing subject cutouts.

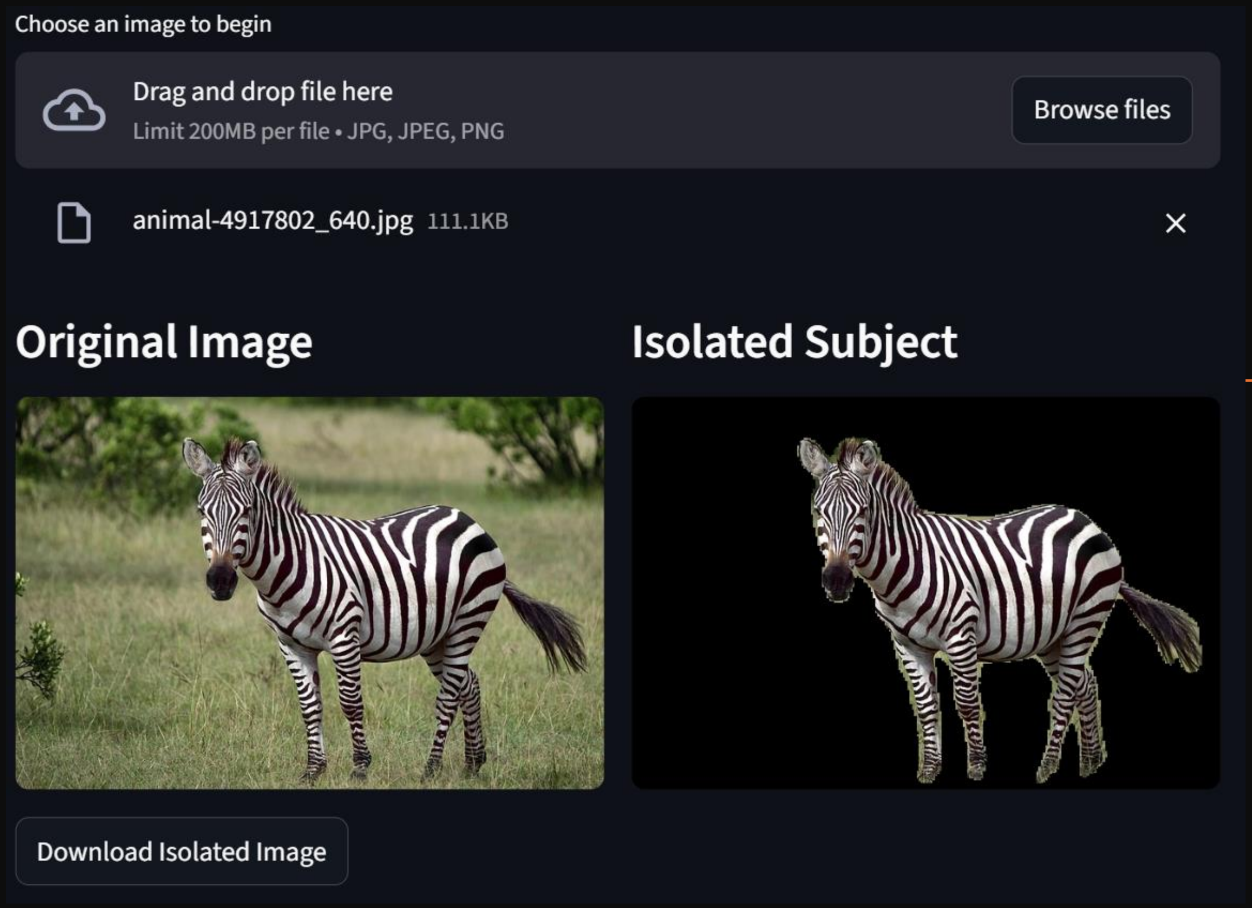
User Interface

To make the model accessible, we developed a web application using **Streamlit**.

Features:

- An intuitive file uploader for JPG and PNG images.
- A side-by-side view of the original uploaded image and the final processed image.
- A download button to save the final image with a transparent background.

Design: The UI was designed to be simple, clean, and user-friendly, closely following the initial sketch.



Challenges

Several challenges were encountered and overcome during the project:

- **Initial Poor Performance:** The first U-Net model trained from scratch produced noisy and inaccurate masks, highlighting the need for transfer learning.
- **TensorFlow/PyTorch Errors:** We encountered and resolved several framework-specific issues, such as TensorFlow graph execution errors and PyTorch DataLoader crashes in multi-worker setups.
- **Streamlit Dependencies:** Setting up the interactive Streamlit app required resolving dependency conflicts between Streamlit and its charting libraries (altair)

Future Scope

- Post-Processing: Implement Conditional Random Fields (CRF) as a post-processing step to further refine mask edges and create even sharper cutouts.
- Larger Dataset: Train the model on the full COCO train2017 dataset (118,000 images) for potentially even better performance and robustness.
- Advanced Architectures: Experiment with state-of-the-art models like DeepLabV3+ to compare performance and potentially gain further improvements.

Conclusion

The VisionExtract project successfully achieved its goal of creating a high-performance model for automatic subject isolation. By leveraging a **U-Net architecture with a pre-trained ResNet34 encoder**, we achieved a Dice score of over 93% on the complex COCO dataset. The final model was deployed in a user-friendly Streamlit application, demonstrating a complete end-to-end pipeline from data processing to a practical, real-world tool.

Thank You